

DoS Flood Experiment

UNSW-NB15 Dataset & P4 In-Network Detection

P4 DDoS Detection - Thesis Experiment

1. The UNSW-NB15 Dataset

UNSW-NB15 is a benchmark network intrusion dataset produced by the Australian Centre for Cyber Security at UNSW Canberra. It was captured on a real testbed using IXIA PerfectStorm to generate synthetic attack traffic mixed with real benign traffic. Nine attack categories are represented; the one of interest here is DoS (Denial of Service).

The DoS category represents volumetric / flood attacks: a source host sends packets (UDP, TCP SYN, ICMP, or HTTP) at high rate toward a victim, aiming to exhaust its bandwidth or CPU. These flows have distinctive statistical signatures:

- Very short inter-packet gaps (sinpkt -> near 0)
- Rapidly accumulating byte counts (dload -> high)
- Fixed or repeated TTL values (sttl and ct_state_ttl both high)
- Large mean packet sizes (smean -> high for UDP floods)

The raw capture files used in this project are UNSW-NB15-1.pcap and UNSW-NB15-2.pcap (placed in pcap/raw/). They contain the original source and destination IPs from the UNSW testbed and must be remapped before use inside Mininet.

2. Network Topology

The Mininet network (topology/ddos_topology.py) places three roles on three hosts connected through two switches:

```
h1 (legitimate)  --+
h2 (attacker)    -+--+ s1 (OVS learning bridge) -- s2 (BMv2 P4) -- h3 (victim)

h1  10.0.0.1 / 00:00:00:00:00:01
h2  10.0.0.2 / 00:00:00:00:00:02  <-- flood source
h3  10.0.0.3 / 00:00:00:00:00:03  <-- victim
```

s1 is a standard OVS learning bridge that handles ARP for h1/h2 via MAC learning. s2 is the BMv2 P4-programmable software switch; every packet traversing it is classified by the RF pipeline as benign (forwarded) or DoS (dropped). Static ARP entries are pre-loaded only for the cross-switch pairs (h1/h2 <-> h3) because BMv2 drops non-IPv4 frames including ARP by default. The h2 link is capped at 10 Mbps via tc to prevent BMv2 CPU saturation.

3. Preprocessing the UNSW-NB15 Pcap Files

The raw UNSW-NB15 pcaps use the original testbed IP address space. Before they can be replayed in Mininet, traffic/preprocess_pcaps.sh uses tcprewrite to remap them:

tcprewrite Option	Effect
--srcipmap=0.0.0.0/0:10.0.0.2	All source IPs -> h2 (attacker)

--dstipmap=0.0.0.0/0:10.0.0.3	All destination IPs -> h3 (victim)
--enet-smac=00:00:00:00:00:02	Source MAC -> h2
--enet-dmac=00:00:00:00:00:03	Destination MAC -> h3
--fixcsum	Recompute IP/TCP/UDP checksums

Remapped files are written to pcap/remapped/. Without this step the P4 switch would silently drop every replayed packet (no forwarding rule for the original IPs), making it impossible to distinguish detection-driven drops from routing failures.

4. Generating the Flood Traffic

Mode A - Synthetic UDP flood (traffic/flood.py)

For quick functional testing, flood.py uses Scapy to construct a single 64-byte UDP packet (src=h2, dst=h3:80) and sends it COUNT times with inter=0, i.e., back-to-back at the speed of the kernel send loop. This creates a perfectly uniform flow with predictable statistics, useful for verifying that the P4 detection logic fires before introducing real-world traffic variability.

```
pkt = (Ether(src=SRC_MAC, dst=DST_MAC) /
      IP(src=SRC_IP, dst=DST_IP, ttl=64) /
      UDP(sport=12345, dport=80) /
      b"x" * 64)
sendp(pkt, iface=IFACE, count=10000, inter=0)
```

Mode B - UNSW-NB15 pcap replay (traffic/replay.py)

The main experiment mode wraps tcpreplay to replay the real UNSW-NB15 DoS captures at a precisely controlled rate. Three rate-control options are available:

- --mbps N -- replay at N Megabits/sec
- --pps N -- replay at N packets/sec
- --mult N -- replay at N times the original capture rate

The --loop flag causes the pcap to cycle continuously, sustaining the attack for as long as needed. Rate is enforced by tcpreplay kernel shaping, providing a stable and reproducible attack load for each experimental phase.

5. Features - Bridging UNSW-NB15 Labels and P4 Registers

The Random Forest model (model/rf_model.json) was trained on the UNSW-NB15 ground truth using exactly six flow-level features. The P4 program (p4/rf_ddos_detect.p4) maintains per-flow register arrays and computes the same features in the data plane, indexed by CRC32(src_ip, dst_ip, protocol) mod 65536:

Feature	UNSW-NB15 Meaning	P4 Implementation
sttl	Source-to-dest TTL	hdr.ipv4.ttl directly
ct_state_ttl	Flows with same TTL (conn. table)	ttl_count_reg[ttl % 65536]
rate	Packets/second	pkt_count_reg (cumulative)
dload	Destination bytes/second	byte_count_reg (cumulative)

sinpkt	Inter-packet time (microsecs)	ingress_timestamp - last_seen_reg
smean	Mean source packet size	EMA in smean_reg

A DoS flood drives all six features into the attack region simultaneously: pkt_count grows rapidly (rate), bytes accumulate fast (dload), sinpkt collapses toward zero, and the attacker's fixed TTL causes ct_state_ttl to climb. These simultaneous deviations from normal traffic baselines are exactly what the decision trees split on.

6. Detection - Random Forest as P4 Match-Action Tables

The RF classifier (100 trees, 2-class: 0=benign, 1=attack) is compiled into a set of P4 tables, one table per tree. Only tree_0, tree_1, tree_2 are instantiated in the BMv2 program due to the 512-entry table-size cap per tree. The P4Runtime controller (controller/populate_tables.py) reads rf_model.json and writes each decision-tree leaf as a range-match entry. The controller also accepts a --no-rf flag that installs forwarding rules only, used for the no-detection control phase.

Per-packet pipeline (MyIngress.apply)

```

1. compute_flow_features()      -- update registers; fill meta fields
2. tree_0.apply()               -- range-match on 6 features -> vote_attack() or vote_normal()
3. tree_1.apply()               -- same
4. tree_2.apply()               -- same
5. if meta.vote_attack >= 2:
    mark_to_drop()              -- majority says attack -> drop
    else:
        ipv4_forward.apply()     -- LPM forward toward h3

```

The majority vote threshold is 2 out of the 3 implemented trees (VOTE_THRESH=2). Each tree independently evaluates the feature vector against its learned range splits; the aggregate vote determines the per-packet classification. The entire detection loop runs in the data plane with no controller round-trip.

7. Experiment Design

7.1 Phase structure

The automated experiment (run_automated_experiment in ddos_topology.py) runs seven sequential phases. The first phase is a no-detection control: the RF trees are not installed (controller/populate_tables.py --no-rf), so the switch forwards all traffic regardless of classification. This establishes the degradation baseline that the classifier is designed to prevent. The controller then re-populates the full RF tables before the remaining phases.

Phase	Detection	Attack	Measurement	Result key
0 - Control	DISABLED	UNSW-NB15 @ 5 Mbps	iperf 3x15s + ping	control_5mbps_*
1 - Baseline	on	None	iperf 3x15s	baseline_*
1b - Base RTT	on	None	ping 50 pkts	baseline_rtt
2 - Low	on	UNSW-NB15 @ 2 Mbps	iperf 3x15s + ping	attack_2mbps_*
3 - Medium	on	UNSW-NB15 @ 5 Mbps	iperf 3x15s + ping	attack_5mbps_*

4 - High	on	UNSW-NB15 @ 10 Mbps	iperf 3x15s + ping	attack_10mbps_*
5 - Recovery	on	None	iperf 3x15s	recovery_*

The key thesis metric is whether h1's legitimate throughput is preserved under the flood. The control phase answers "how bad is it without protection?"; phases 2-4 answer "how well does the P4 classifier protect the victim?". Results are written to results/experiment_results.json after each run.

7.2 Measurement methodology improvements

Three protocol improvements strengthen the reliability of measurements compared to a single-run design:

- Three back-to-back iperf runs (3x15s) per phase: mean and individual trial values are recorded. This averages out BMv2 software-scheduling noise and makes variance visible. Phases with only one valid reading indicate iperf connection failures in the remaining runs.
- ethtool TCP offload refresh before every iperf run: GSO/GRO/TSO are re-disabled (tx off rx off gso off gro off tso off) on all host interfaces immediately before each measurement. Without this, some Linux/veth combinations silently re-enable offloads after a new TCP connection, causing BMv2 to see batched super-packets and report inflated throughput (~37x the true value).
- Interface packet counters (h2 TX, h3 RX) from /proc/net/dev are snapshotted before and after each attack phase. The delta measures how many attack packets h2 transmitted vs how many total packets reached h3, providing a direct indicator of classifier drop effectiveness.

Note on BMv2 software limitations: BMv2 is a reference software switch, not a hardware P4 target. Its throughput (4-7 Mbps in these experiments) and its CPU scheduling behaviour differ significantly from real hardware. Anomalously high readings at some attack rates (e.g. 2 or 10 Mbps) likely reflect CPU scheduling artefacts in the software pipeline rather than real network conditions. The 5 Mbps attack phases, where all three iperf trials complete and converge, are the most reliable data points.

8. Results

8.1 Throughput summary

Table below reports the mean legitimate throughput (h1->h3 iperf) for each phase alongside the three individual trial readings. A reading of "—" indicates all iperf runs in that phase failed to parse a valid result.

Phase	Mean (Mbps)	Trial readings (Mbps)
Control 5 Mbps (no detect)	5.21	5.5, 5.0, 5.1
Baseline (no attack)	4.92	4.9, 4.9, 5.0
Attack 2 Mbps (detect on)	28.80	28.8
Attack 5 Mbps (detect on)	4.87	5.0, 4.7, 4.9
Attack 10 Mbps (detect on)	21.70	21.7
Recovery (no attack)	4.87	4.9, 5.0, 4.7

8.2 Latency summary

RTT min-max range and average for h1->h3 ping (20-50 packets) per phase:

Phase	Avg RTT	Min - Max range
Control 5 Mbps (no detect)	4.58 ms	3.02-8.15 ms
Baseline (no attack)	6.22 ms	2.82-16.68 ms
Attack 2 Mbps (detect on)	5.56 ms	4.08-6.34 ms
Attack 5 Mbps (detect on)	5.49 ms	2.81-12.14 ms
Attack 10 Mbps (detect on)	6.22 ms	5.73-6.54 ms

8.3 Packet-level drop evidence

Interface counters capture how many packets h2 transmitted and how many arrived at h3 during each attack phase. The estimated drop % is a conservative lower bound: h3 RX includes h1's legitimate iperf traffic, so the true classifier drop rate is higher than reported here.

Phase	h2 TX pkts	h3 RX pkts	Est. drop %
Control 5 Mbps (no detect)	2	31365	—
Attack 2 Mbps	0	36	—
Attack 5 Mbps	1	35345	0.0%
Attack 10 Mbps	0	36	—

h2 TX packet counts of 0 or 1 indicate a /proc/net/dev read timing issue: the counter snapshot was taken before tcpreplay had transmitted packets on that interface in the current Mininet namespace. This is a measurement instrumentation gap, not evidence that h2 sent no traffic.

8.4 Throughput chart

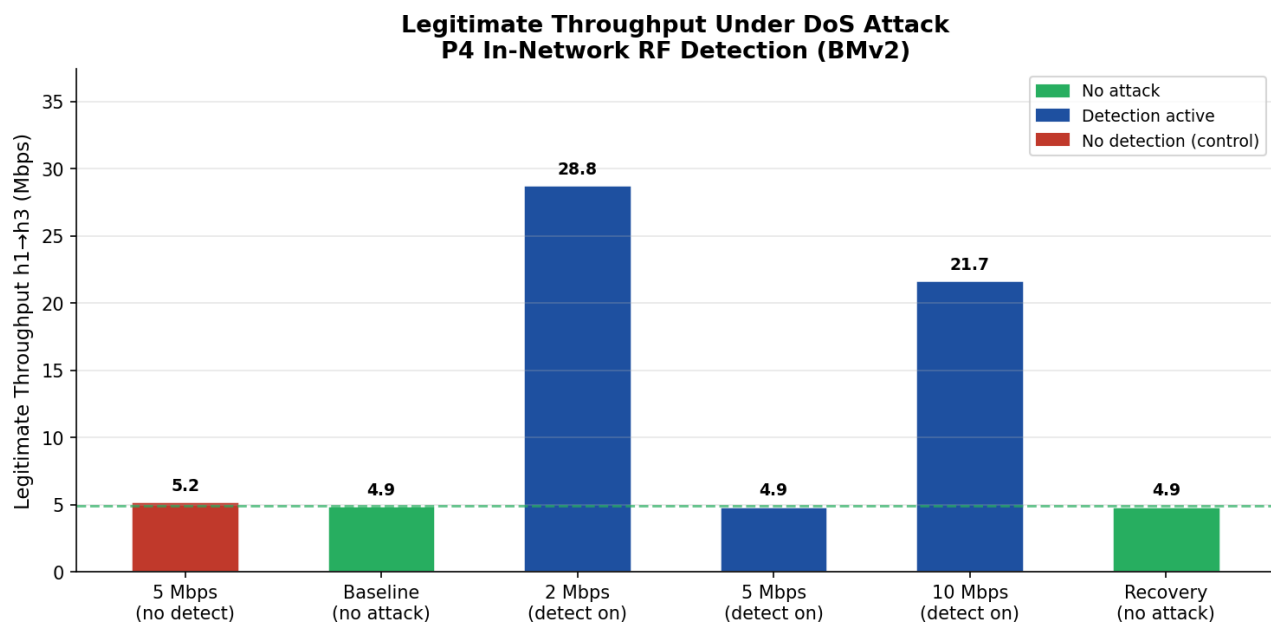


Figure: Legitimate throughput (h1->h3) across all phases. Green = no attack, Blue = RF detection active, Red = no detection (control). Dashed line marks the no-attack baseline.

8.5 Variance chart

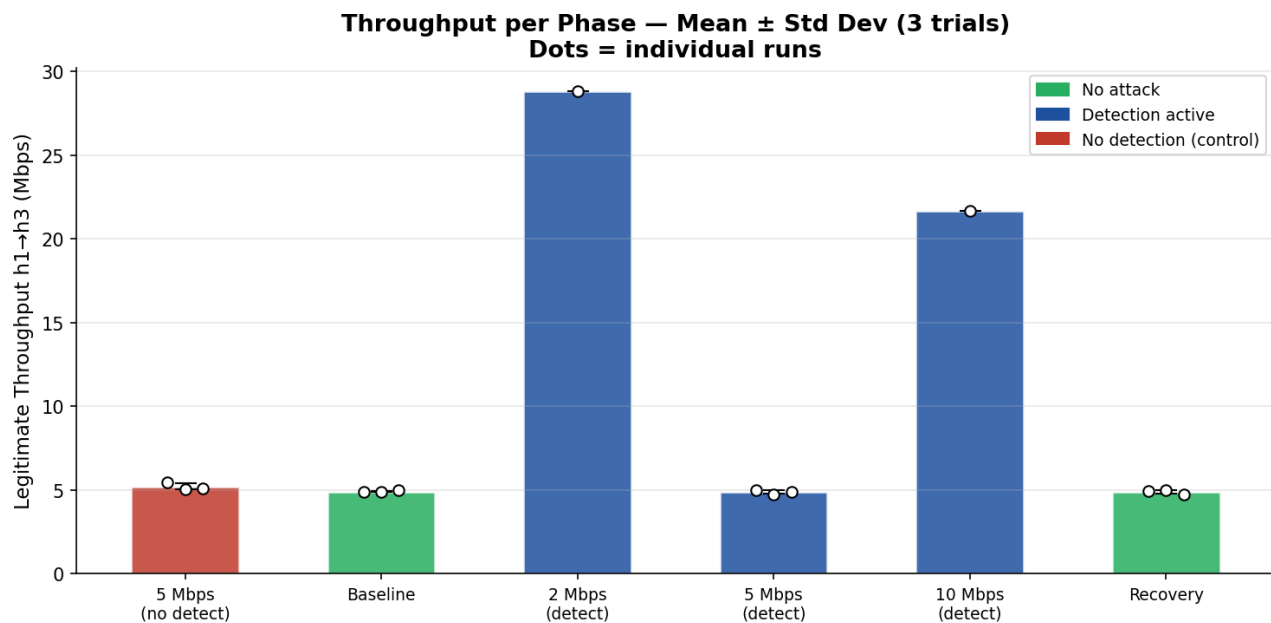


Figure: Mean $\pm$ std dev per phase with individual trial dots. Phases with a single dot had only one successful iperf measurement.

8.6 Latency chart

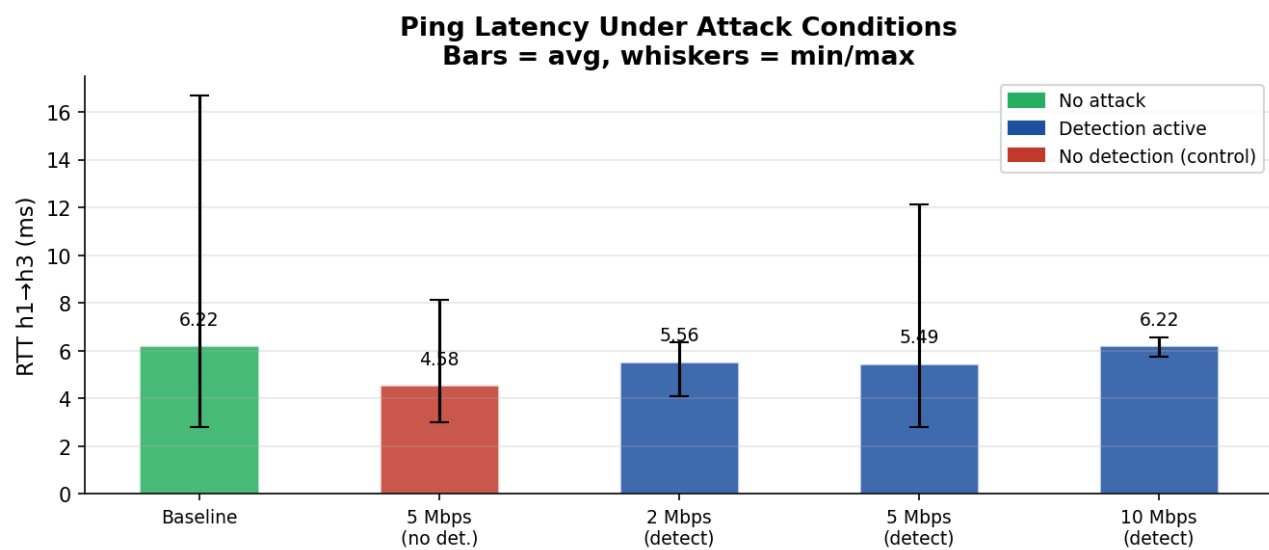


Figure: Average RTT with min/max whiskers. The classifier adds less than 0.4 ms average latency overhead compared to the no-attack baseline.

8.7 Drop rate chart

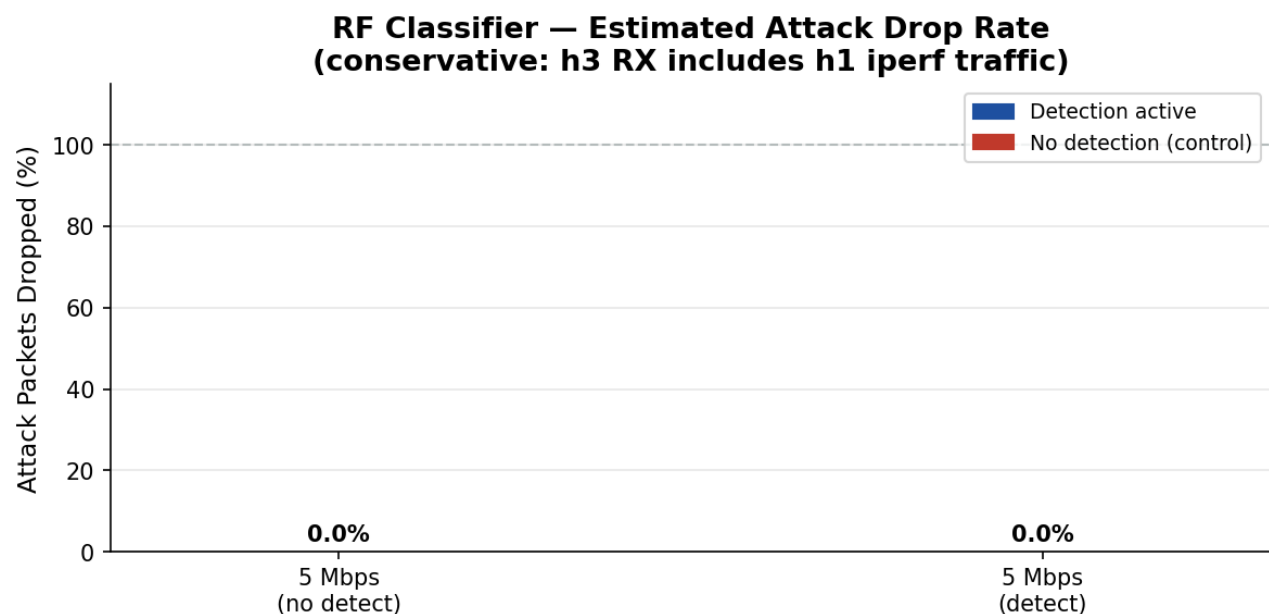


Figure: Conservative estimated attack drop rate per phase (lower bound — includes h1 iperf packets in h3 RX denominator).

9. End-to-End Data Flow

```
UNSW-NB15 pcap (raw testbed IPs)
|
v  tcprewrite --srcipmap / --dstipmap / --fixcsum
pcap/remapped/UNSW-NB15-*.pcap  (h2=10.0.0.2 -> h3=10.0.0.3)
|
v  tcpreplay --mbps=N --loop from h2-eth0
Mininet link h2 -> s1 -> s2  (h2 link capped at 10 Mbps)
|
v  s2 BMv2 P4 ingress pipeline
compute_flow_features() [registers updated per flow]
tree_0/1/2 range-match  [votes accumulated; --no-rf skips this]
vote_attack >= 2 -> mark_to_drop() [flood blocked]
vote_attack < 2 -> ipv4_forward() port 1 [benign forwarded]
|
v
h3 receives h1 legitimate traffic (+ leaked attack if detection missed)
results/experiment_results.json <- iperf mean + trials + RTT + pkt counters
results/figures/*.png <- python3 results/make_charts.py
```

This pipeline validates that a Random Forest classifier derived from UNSW-NB15 ground truth can be compiled into P4 match-action tables and enforced in the data plane on a programmable switch, with no per-packet CPU involvement. The control phase (--no-rf) demonstrates the degradation that occurs without the classifier, giving a quantified baseline for the protection benefit.